

`import`  `try/except`

Módulos y Manejo de Errores en Python

Divide tu código y controla lo inesperado

Skill: Python I

Docente: Jhonathan Arley Peñaloza Sierra

Campuslands · 2026

Objetivos de Aprendizaje

1 Comprender qué es la modularidad y por qué es importante

2 Crear e importar módulos propios en Python

3 Conocer iterables, `iter()` y `next()`

4 Entender la diferencia entre errores y excepciones

5 Dominar `try-except`, `else` y `finally`

¿Qué es la Modularidad?

Piensa en bloques LEGO...

Cada pieza de LEGO:

- Es independiente
- Se puede reusar en otro modelo
- Tiene una función específica
- Se conecta con otras piezas

¡Construyes algo grande con piezas pequeñas!

Un módulo en Python es igual...

Un módulo:

- Es un archivo .py independiente
- Se puede reusar en otro proyecto
- Contiene funciones específicas
- Se conecta con import

¡Construyes un programa grande con módulos pequeños!



Definición:

La modularidad es dividir tu programa en archivos .py más pequeños (módulos), cada uno con funcionalidad específica. Se reutilizan con import.

Ventajas: Comprensión • Reutilización • Mantenimiento fácil • Colaboración

Importar Módulos en Python

Python usa import para traer código de otros archivos:

import completo

```
import mi_modulo  
  
mi_modulo.procesar(12,345)
```

Importa todo. Usas modulo.funcion()

from...import función

```
from mi_modulo import procesar  
  
procesar(12,345)
```

Importa solo una función.

from...import *

```
from mi_modulo import *  
  
procesar(12,345)
```

Importa todo. Funciones directo.

import con alias

```
from entities import Admin as A  
  
usuario = A()
```

Renombra para evitar conflictos.

Estructura de Archivos y Alias

Los módulos son archivos .py organizados en carpetas:

Misma carpeta:

```
tiendaOnline/  
├── menu.py  
└── main.py
```

```
# En main.py  
from menu import *
```

En subcarpeta:

```
registradora/  
├── registrar.py  
└── tiendas/  
    └── mi_modulo.py
```

```
# En registrar.py  
from tiendas.mi_modulo  
import *
```

Alias (renombrar):

```
from entities import Admin as EntitieAdmin # Renombra para evitar conflictos  
  
# R1: Evitar nombres duplicados R2: Nombre más apropiado al contexto
```

Ejemplo: Módulo de Menú

 tiendaOnline/ → menu.py + main.py

menu.py (el módulo):

```
def menu(a, b):  
    print("Menu:")  
    print(a)  
    print(b)  
    while True:  
        nsel = input("Opción: ")  
        if nsel == "1":  
            print("Gracias!")  
            return nsel  
        elif nsel == "2":  
            print("Vuelva pronto!")  
            return nsel  
        else:  
            print("Opción inválida")
```

main.py (programa principal):

```
from menu import *  
  
print("Hola!!")  
op1 = "1: Comprar Revista"  
op2 = "2: No comprar"  
  
nsel = menu(op1, op2)  
  
if nsel == "1":  
    print("Elegiste comprar")  
else:  
    print("No compraste nada")
```



El módulo menu.py se crea UNA vez y se puede usar en cualquier proyecto.

Iterables: iter() y next()

Un iterable es cualquier objeto que se puede recorrer (cadenas, listas, tuplas, dicts).

```
repisa = ["Peluche","Libro","celular","lampara"]
objetos = iter(repisa)    # Crea el iterador

print(next(objetos))      # → Peluche
print(next(objetos))      # → Libro
print(next(objetos))      # → celular
print(next(objetos))      # → lampara

# Ejemplo 2: cadenas
mi_cadena = "Casa"
it = iter(mi_cadena)
print(next(it))  # C
print(next(it))  # a
```

Ventajas:

- Consume menos memoria
- Permite lógica diferente a while o for
- Avanza uno a uno bajo tu control

⚠ Si llamas next() más veces que elementos, da StopIteration error.

Errores vs Excepciones

Ambos interrumpen tu programa, pero son diferentes:

Errores

Problemas en el código que impiden que se ejecute correctamente.

- Sintácticos: código mal escrito
- Semánticos: lógica incorrecta

Deben corregirse en el código.



Excepciones

Eventos inesperados durante la ejecución del programa.

- Dividir por cero
- Archivo que no existe
- Tipo de dato incorrecto

Se manejan con try-except.



Los errores se corrigen editando código. Las excepciones se manejan con try-except.

Excepciones más comunes

TypeError

Tipo incorrecto

```
5 + "2"
```

ValueError

Valor no válido

```
int("abc")
```

ZeroDivisionError

Dividir entre cero

```
10 / 0
```

IndexError

Índice fuera de rango

```
lista[99]
```

FileNotFoundError

Archivo no encontrado

```
open("no.txt")
```

KeyError

Llave no existe

```
dict["no"]
```

¿Cómo se ve un error en consola?

Cuando ocurre una excepción, Python muestra un Traceback:

```
Traceback (most recent call last):  
  File "<nombre del archivo>", line <número de línea>, in <module>  
    element = my_list[3]  
IndexError: list index out of range
```

El Traceback te dice:

- En qué archivo ocurrió
- En qué línea está el problema
- Qué tipo de excepción es
- Un mensaje descriptivo



¡No le tengas miedo al Traceback! Es tu mejor amigo. Léelo de abajo hacia arriba.

try-except: Manejar Excepciones

Con try-except capturas errores y evitas que tu programa se detenga.

Sintaxis:

```
try:
    # Código que puede fallar
except TipoDeError:
    # Qué hacer si falla
```

Ejemplo:

```
x = 5; y = "2"
try:
    result = x + y
except TypeError:
    print("Valores deben
          ser numéricos")
```

Resultado:

```
PS C:\Users\User\Desktop\clasedemo3> c::; cd 'c:\Users\User\Desktop\clasedemo3'; & 'C:\Users\User\AppData\Local\Programs\Python\Python311\python.exe'
' 'c:\Users\User\.vscode\extensions\ms-python.python-2023.14.0\pythonFiles\lib\python\debugpy\adapter\..\..\debugpy\launcher' '52762' '--' 'c:\Users
\User\Desktop\clasedemo3\precios.py'
El valor de los sumandos deben ser numéricos
PS C:\Users\User\Desktop\clasedemo3> █
```

 Sin try-except el programa se detiene. Con try-except capturamos el error y mostramos un mensaje.

try-except Anidados

Puedes capturar DIFERENTES tipos de excepciones:

```
try:
    try:
        n1 = int(input("Primer número: "))
        n2 = int(input("Segundo número: "))
        resultado = n1 / n2
        print("Resultado:", resultado)
    except ValueError:
        print("Valor incorrecto")
    except ZeroDivisionError:
        print("No dividir por cero")
except Exception as e:
    print(e)
```

Resultados:

```
PS C:\Users\User\Desktop\clasedemo3> cd 'c:\Users\User\Desktop\clasedemo3'; & 'C:\Users\User\AppData\Local\Programs\Python\Python311\python.exe'
'c:\Users\User\.vscode\extensions\ms-python.python-2023.14.0\pythonFiles\lib\python\debugpy\adapter\..\..\Vdebugpy\launcher' '53527' '-' 'c:\Users
\User\Desktop\clasedemo3\precios.py'
Digite el primer número: 45
Digite el segundo número: 0
No se puede dividir por cero
PS C:\Users\User\Desktop\clasedemo3> cd 'c:\Users\User\Desktop\clasedemo3'; & 'C:\Users\User\AppData\Local\Programs\Python\Python311\python.exe'
'c:\Users\User\.vscode\extensions\ms-python.python-2023.14.0\pythonFiles\lib\python\debugpy\adapter\..\..\Vdebugpy\launcher' '53537' '-' 'c:\Users
\User\Desktop\clasedemo3\precios.py'
Digite el primer número: 14
Digite el segundo número: asd
Algun numero tiene valor incorrecto
PS C:\Users\User\Desktop\clasedemo3> |
```

```
PS C:\Users\User\Desktop\clasedemo3> cd 'c:\Users\User\Desktop\clasedemo3'; & 'C:\Users\User\AppData\Local\Programs\Python\Python311\python.exe'
'c:\Users\User\.vscode\extensions\ms-python.python-2023.14.0\pythonFiles\lib\python\debugpy\adapter\..\..\Vdebugpy\launcher' '54158' '-' 'c:\Users
\User\Desktop\clasedemo3\precios.py'
Digite el primer número: 90
Digite el segundo número: 3
El resultado de la division es: 30.0
PS C:\Users\User\Desktop\clasedemo3> |
```



El bloque interno captura errores específicos. El externo captura cualquier otra excepción.

Bloques else y finally

else — Si NO hubo error:

```
try:
    n1 = int(input("Número 1: "))
    n2 = int(input("Número 2: "))
    res = n1 / n2
except ValueError:
    print("Valor incorrecto")
except ZeroDivisionError:
    print("No dividir por 0")
else:
    print("Resultado:", res)
```

*else se ejecuta SOLO si no
hubo ninguna excepción.*

finally — SIEMPRE se ejecuta:

```
file = None
try:
    file = open("archivo.txt", "r")
    contenido = file.read()
except FileNotFoundError:
    print("No se encontró")
finally:
    if file:
        file.close()
    print("Proceso terminado")
```

*finally se ejecuta SIEMPRE,
haya o no error.*

```
PS C:\Users\User\Desktop\clasedemo3> c::; cd 'c:\Users\User\Desktop\clasedemo3'; & 'C:\Users\User\AppData\Local\Programs\Python\Python311\python.exe'
' 'c:\Users\User\.vscode\extensions\ms-python.python-2023.14.0\pythonFiles\lib\python\debugpy\adapter\..\..\debugpy\launcher' '54473' '--' 'c:\Users
\User\Desktop\clasedemo3\precios.py'
No se encontró el archivo.
PS C:\Users\User\Desktop\clasedemo3>
```



else = cuando todo sale bien. finally = SIEMPRE (ideal para cerrar archivos).

Estructura completa

```
try:
    # Código que puede fallar
    resultado = operacion()

except ValueError:
    print("Valor inválido")

except ZeroDivisionError:
    print("No dividir por cero")

except Exception as e:
    print("Error:", e)

else:
    # Si NO hubo error
    print("Todo bien:", resultado)

finally:
    # SIEMPRE se ejecuta
    print("Proceso terminado")
```

Flujo:

- 1 try**
Intenta ejecutar
- 2 except**
Si hay error, lo captura
- 3 else**
Si NO hubo error
- 4 finally**
SIEMPRE se ejecuta



Actividad: Módulos

1. Crea "calculadora.py" con funciones: sumar, restar, multiplicar, dividir
2. Crea "main.py" que importe calculadora.py
3. Desde main.py, llama cada función con diferentes números
4. Prueba importar con: import, from...import, y alias



Tiempo: 10 minutos



Actividad: Manejo de Errores

1. Pide 2 números y divídelos
2. Agrega try-except para ValueError
3. Agrega except para ZeroDivisionError
4. Agrega else para mostrar resultado
5. Agrega finally con "Programa terminado"
6. Intenta abrir un archivo que no existe con try-except



Tiempo: 15 minutos

Hoja de Referencia Rápida

Operación	Código
Importar módulo	<code>import mi_modulo</code>
Importar función	<code>from mi_modulo import funcion</code>
Importar todo	<code>from mi_modulo import *</code>
Alias	<code>import modulo as m</code>
Capturar error	<code>try: ... except Error: ...</code>
Múltiples except	<code>except ValueError: ... except ZeroDivision...</code>
Si no hay error	<code>else: # después de except</code>
Siempre ejecutar	<code>finally: # siempre se ejecuta</code>

Errores Comunes de Principiantes

except muy general

Causa: Usar except: sin tipo

Solución: Especifica: except ValueError:

except: pass

Causa: No hacer nada con el error

Solución: Al menos imprime un mensaje.

No cerrar archivos

Causa: Olvidar .close()

Solución: Usa with open() o finally

Importar mal

Causa: Archivo no en la ruta correcta

Solución: Verifica carpetas y usa puntos.



Resumen de la Clase



Modularidad: divide tu programa en archivos .py independientes



import, from...import, alias — formas de traer módulos



iter() y next() recorren iterables sin for/while



Errores (código mal) vs Excepciones (eventos inesperados)



try-except captura excepciones sin detener el programa



else = si no hay error, finally = SIEMPRE se ejecuta

¡Ahora puedes organizar tu código y manejar errores como un profesional! 🚀

¿Preguntas?

*"Divide tu código en módulos
y maneja los errores con elegancia"*